

SAMVAADA AI

Verified Multilingual Voice Intelligence for the 1092 Citizen Helpline

Comprehensive Project Implementation Plan

1. Executive Summary

Samvaada AI is a production-ready, multilingual voice intelligence platform designed to handle the Karnataka 1092 citizen helpline. This document provides the complete technical specification, architecture, API contracts, file structure, component hierarchy, and implementation roadmap for full-stack development.

The system is built on a verification-first principle, ensuring accurate understanding before any action is taken. It processes Kannada dialects, code-mixed Hindi/English, and provides real-time assistance to human agents.

2. System Architecture

2.1 High-Level Architecture

The system follows a microservices architecture with clear separation of concerns:

- Voice Input Layer: Telephony integration, audio preprocessing
- Speech Processing: ASR (Whisper), Language ID, Dialect Detection
- NLU Layer: Intent extraction, entity recognition, sentiment analysis
- Verification Engine: Generate verification statement, handle caller confirmation
- Agent Dashboard: Real-time transcript, verified summary, sentiment indicators
- Backend Services: FastAPI servers, Kafka message queue, PostgreSQL database

2.2 Technology Stack

Component	Technology	Version/Notes
Speech Recognition	Whisper (OpenAI)	v3 + fine-tuning for Kannada
Language ID	FastText or LID model	Token-level mixing
NLU	IndicBERT + Custom NER	Kannada & code-mixed
Sentiment Analysis	Kannada BERT / Custom	Speech & text-based
Text-to-Speech	IndicTTS	Kannada with prosody
Backend API	FastAPI	Python 3.10+
Message Queue	Apache Kafka	Event-driven streaming
Database	PostgreSQL + pgvector	Vector search for context

Caching	Redis	Session & model cache
Frontend	React 18 + TypeScript	Shadcn/UI components
Real-time Comm	WebSocket / Socket.io	Agent-to-Server
Deployment	Docker + Kubernetes	Cloud-native

3. API Specifications

3.1 initiate

Endpoint: POST /api/v1/calls/initiate

Purpose: Initialize a new helpline call

Returns: Call session ID, WebSocket connection URL

3.2 {call_id}

Endpoint: wss://api.samvaada.io/ws/call/{call_id}

Purpose: WebSocket Audio Stream

Returns: Binary audio frames (PCM 16-bit, 16kHz)

3.3 process

Endpoint: POST /api/v1/asr/process

Purpose: Process speech to text

Returns: Transcript with confidence

3.4 extract

Endpoint: POST /api/v1/nlu/extract

Purpose: Extract intent and entities

Returns: Intent, entities, confidence

4. Database Schema

4.1 Core Tables

calls (11 fields)

Field	Type	Constraints
call_id	UUID	PRIMARY KEY
phone_number	VARCHAR(15)	HASHED
call_timestamp	TIMESTAMP	NOT NULL
duration_seconds	INTEGER	NULLABLE

status	ENUM	ACTIVE/COMPLETED/FAILED
language	VARCHAR(5)	kn/hi/en
dialect	VARCHAR(50)	NULLABLE
verified	BOOLEAN	DEFAULT false
agent_id	UUID FK	NULLABLE
resolved	BOOLEAN	DEFAULT false
created_at	TIMESTAMP	DEFAULT NOW()

call_transcripts (7 fields)

Field	Type	Constraints
transcript_id	UUID	PRIMARY KEY
call_id	UUID FK	REFERENCES calls
speaker	ENUM	CALLER/AGENT/SYSTEM
text	TEXT	NOT NULL
timestamp	TIMESTAMP	NOT NULL
confidence	FLOAT	0.0-1.0
language	VARCHAR(5)	kn/hi/en

call_nlu (7 fields)

Field	Type	Constraints
nlu_id	UUID	PRIMARY KEY
call_id	UUID FK	REFERENCES calls
intent	VARCHAR(100)	NOT NULL
entities	JSONB	NOT NULL
confidence	FLOAT	0.0-1.0
verified_at	TIMESTAMP	NULLABLE
verification_text	TEXT	NULLABLE

call_sentiment (7 fields)

Field	Type	Constraints
sentiment_id	UUID	PRIMARY KEY
call_id	UUID FK	REFERENCES calls
emotion	VARCHAR(50)	frustrated/angry/sad
urgency_level	ENUM	LOW/MEDIUM/HIGH
anger_score	FLOAT	0.0-1.0
distress_score	FLOAT	0.0-1.0
timestamp	TIMESTAMP	NOT NULL

agents (7 fields)

Field	Type	Constraints
agent_id	UUID	PRIMARY KEY
username	VARCHAR(50)	UNIQUE
email	VARCHAR(100)	UNIQUE
password_hash	VARCHAR(255)	HASHED
status	ENUM	AVAILABLE/BUSY/OFFLINE
calls_handled	INTEGER	DEFAULT 0
created_at	TIMESTAMP	DEFAULT NOW()

5. Project File Structure

Complete hierarchical directory structure with all backend services, frontend components, ML models, and infrastructure code.

```

1. samvaada-ai/
├── backend/                # FastAPI services
│   ├── app/
│   │   ├── main.py        # Entry point
│   │   ├── config.py      # Config management
│   │   ├── database.py    # PostgreSQL setup
│   │   ├── redis_client.py # Redis caching
│   │   ├── kafka_producer.py # Event streaming
│   │   ├── middleware/    # Auth, logging, errors
│   │   ├── routes/        # API endpoints
│   │   ├── websocket/     # WebSocket handlers
│   │   ├── services/      # Business logic
│   │   ├── models/        # ORM & schemas
│   │   ├── ml_models/     # ML model wrappers
│   │   └── utils/         # Helpers
│   ├── tests/
│   ├── migrations/        # Alembic DB migrations
│   ├── requirements.txt
│   ├── Dockerfile
│   └── docker-compose.yml
├── frontend/              # React dashboard
│   ├── src/
│   │   ├── pages/         # Login, Dashboard, History
│   │   ├── components/    # UI components
│   │   ├── hooks/         # Custom hooks
│   │   ├── services/      # API client
│   │   ├── store/         # Redux state
│   │   └── styles/        # CSS

```

```

| | | types/          # TypeScript types
| | | └─ utils/
| | └─ package.json
| | └─ tsconfig.json
| | └─ vite.config.ts
| | └─ Dockerfile
| | └─ .env.example
|
└─ ml-models/          # Model weights & training
    └─ asr/
    └─ nlu/
    └─ sentiment/
    └─ language_id/
    └─ training_scripts/
|
└─ infrastructure/     # DevOps
    └─ kubernetes/
    └─ docker/
    └─ terraform/
    └─ ansible/
    └─ monitoring/
|
└─ documentation/
└─ scripts/
└─ config/
└─ .github/workflows/
└─ docker-compose.yml

```

6. Component Interactions & Data Flow

6.1 Call Initiation Flow

2. Caller dials 1092 → Telephony gateway generates call_id
3. POST /api/v1/calls/initiate → Backend creates call record
4. Backend returns WebSocket URL + session token
5. Caller device establishes WS connection (PCM audio)
6. WebSocket handler subscribes to Kafka topic

6.2 Real-Time Speech Processing

7. Audio frames arrive via WebSocket → buffer in manager
8. ASR Service processes buffered audio (500ms chunks)
9. Whisper transcribes → Language ID detects language
10. Partial transcript sent back via WS (real-time feedback)
11. Full transcript stored in call_transcripts table

12. Published to Kafka for NLU processing

6.3 Intent & Entity Extraction

13. NLU Service consumes transcript from Kafka

14. IndicBERT-based NER extracts entities

15. Intent classifier predicts action category

16. Results stored in call_nlu table

17. Published to verification-input Kafka topic

6.4 Verification Loop

18. Verification Service generates statement

19. TTS Service converts to Kannada speech

20. Audio sent to caller via WS

21. Caller responds YES/NO → captured as confirmation

22. Verification status updated with timestamp

6.5 Sentiment Analysis

23. Sentiment Service processes audio & transcript in parallel

24. Speech-based: Prosody analyzer detects emotion

25. Text-based: Kannada BERT classifier

26. Scores stored in call_sentiment table

27. Urgency level determined (LOW/MEDIUM/HIGH)

6.6 Agent Dashboard Real-Time Update

28. Agent dashboard polls GET /api/v1/calls/{call_id}/context

29. Backend aggregates from multiple tables

30. Redis caches context for 30 seconds

31. Agent sees: verified summary, entities, emotion, urgency

32. Agent resolves issue → call marked COMPLETED

7. Core Service Specifications

7.1 ASR Service

File: app/services/asr_service.py

Responsibilities:

- Load Whisper model (quantized)
- Process audio chunks (PCM 16-bit, 16kHz)
- Detect language (Kannada/Hindi/English)
- Identify dialect (Bangalore, Hubli, etc.)
- Return transcript with confidence

Key Methods:

- `process_audio(audio_bytes) → Dict`
- `detect_language(text) → str`
- `identify_dialect(text, language) → str`

7.2 NLU Service

File: `app/services/nlu_service.py`

Responsibilities:

- Extract named entities (location, person, issue)
- Classify intent (complaint, inquiry, request)
- Handle code-mixed input (Kannada + Hindi/English)
- Confidence scoring for extractions

Key Methods:

- `extract_entities(text, language) → Dict`
- `classify_intent(text) → str`
- `handle_code_mixed(text) → List[Token]`

7.3 Verification Service

File: `app/services/verification_service.py`

Responsibilities:

- Generate verification statements
- Handle edge cases (multiple entities)
- Manage caller confirmation logic
- Ensure natural Kannada phrasing

Key Methods:

- `generate_verification_text(intent, entities) → str`
- `handle_confirmation(confirmed, call_id) → bool`
- `refine_on_rejection(call_id) → str`

7.4 Sentiment Service

File: `app/services/sentiment_service.py`

Responsibilities:

- Analyze speech prosody (anger, frustration)
- Analyze text sentiment (negative, positive)
- Assign urgency level
- Alert system for high-urgency calls

Key Methods:

- `analyze_speech(audio_bytes) → Dict`
- `analyze_text(text) → Dict`
- `compute_urgency_score() → str`

8. Implementation Timeline

Phase 1: Foundation & Setup (Weeks 1-3)

- Set up project repo structure
- Configure PostgreSQL + Redis
- Setup FastAPI scaffold
- Create database ORM models
- Initialize React project

Phase 2: Core Services (Weeks 4-7)

- Implement ASR Service
- Implement NLU Service
- Implement Sentiment Service
- Implement TTS Service
- Unit tests for each service

Phase 3: API & WebSocket (Weeks 8-9)

- Implement REST API endpoints
- Implement WebSocket manager
- Implement Kafka producer/consumer
- Integration tests

Phase 4: Agent Dashboard (Weeks 10-11)

- Build React components
- Implement authentication
- Connect to WebSocket
- UI/UX testing

Phase 5: Testing & Refinement (Weeks 12-13)

- End-to-end testing
- Adversarial testing
- Performance benchmarking
- Security audit

Phase 6: Deployment (Week 14)

- Containerize services (Docker)

- Deploy to Kubernetes
- Set up monitoring
- Production launch

9. Security & Compliance

9.1 PII Masking & Privacy

- Phone numbers: SHA-256 hash for storage
- Addresses: Generalized to district/ward level
- Audio retention: 7-day TTL, auto-delete
- Transcripts: Anonymized in logs

9.2 Encryption

- In-transit: TLS 1.3 for HTTPS/WSS
- At-rest: AES-256 for database
- API Keys: Hashed, rotate every 90 days

9.3 Authentication & Authorization

- JWT-based API authentication (HS256)
- Role-based access control (RBAC)
- Multi-factor authentication for admins
- Session timeout: 8 hours for agents

10. Monitoring & Observability

10.1 Metrics (Prometheus)

- API response times (p50, p95, p99)
- ASR latency per call
- Database query performance
- Kafka topic lag
- Error rates & exceptions

10.2 Visualization (Grafana)

- Real-time call status dashboard
- Agent performance metrics
- System health overview
- Sentiment analytics by hour/day

11. Key Performance Indicators (KPIs)

Metric	Target	Measurement
--------	--------	-------------

Call Handling Time	-40%	Avg call duration (after vs before)
Verification Accuracy	>95%	Correctly understood on first try
ASR WER	<15%	Word error rate on Kannada test set
System Uptime	>99.5%	Production availability SLA
End-to-End Latency	<3s	Time from utterance to agent context
Agent Satisfaction	>4/5	User surveys on AI assistance
Citizen Trust	+30%	Post-call survey: understood correctly
Cost Reduction	-25%	Per-call operational cost vs manual

Conclusion

This comprehensive plan provides a complete roadmap for implementing Samvaada AI as a production-grade platform. The verification-first interaction model, combined with multilingual capabilities and human-in-the-loop design, addresses the core pain points of the 1092 helpline system.

Implementation should follow the phased approach outlined, with rigorous testing at each stage. The modular architecture allows for iterative deployment and continuous improvement based on real-world feedback.